

C++学习笔记2

2022年3月19日 20:45

- 标准库

- 命名空间的using声明

- std: : cin,

- std: : 使用std空间中的名字cin

- using举例

- using namespace: : std 使用std命名空间

- 每个名字都需要独立的using声明

```
#include <iostream>
// 通过下列 using 声明，我们可以使用标准库中的名字
using std::cin;
using std::cout; using std::endl;
int main()
{
    cout << "Enter two numbers:" << endl;
```

- 注意，用到的每个名字都必须有自己的声明语句，而且每句话都得以分号结束。

- 头文件不包括using声明

- string标准库

- 初始化string

```
string s1;                // 默认初始化，s1 是一个空字符串
string s2 = s1;           // s2 是 s1 的副本
string s3 = "hiya";       // s3 是该字符串字面值的副本
string s4(10, 'c');       // s4 的内容是 cccccccccc
```

表 3.1: 初始化 string 对象的方式

string s1	默认初始化，s1 是一个空串
string s2(s1)	s2 是 s1 的副本
string s2 = s1	等价于 s2(s1)，s2 是 s1 的副本
string s3("value")	s3 是字面值"value"的副本，除了字面值最后的那个空字符外
string s3 = "value"	等价于 s3("value")，s3 是字面值"value"的副本
string s4(n, 'c')	把 s4 初始化为由连续 n 个字符 c 组成的串

- 拷贝初始化和直接初始化

- 如果使用等号(=)初始化一个变量，实际上执行的是拷贝初始化(copy initialization)，编译器把等号右侧的初始值拷贝到新创建的对象中去。与之相反，如果不使用等号，则执行的是直接初始化(direct initialization)。

- 读取string的操作

表 3.2: string 的操作	
os<<s	将 s 写到输出流 os 当中, 返回 os
is>>s	从 is 中读取字符串赋给 s, 字符串以空白分隔, 返回 is
getline(is, s)	从 is 中读取一行赋给 s, 返回 is
s.empty()	s 为空返回 true, 否则返回 false
s.size()	返回 s 中字符的个数
s[n]	返回 s 中第 n 个字符的引用, 位置 n 从 0 计起
s1+s2	返回 s1 和 s2 连接后的结果
s1=s2	用 s2 的副本代替 s1 中原来的字符
s1==s2	如果 s1 和 s2 中所含的字符完全一样, 则它们相等; string 对象的相等性判断对字母的大小写敏感
s1!=s2	等性判断对字母的大小写敏感
<, <=, >, >=	利用字符在字典中的顺序进行比较, 且对字母的大小写敏感

□ 注意

- ◆ 在执行读取操作时, cin, os, 等等时候string对象会自动忽略开头的空白（即空格符、换行符、制表符等）并从第一个真正的字符开始读起, 直到遇见下一处空白为止。
- ◆ string和和字符串数组有哪些区别: string相当于一个没有下标的字符串, 其表示方法是一样的
- ◆ 拷贝初始化和直接初始化
 - ◇ 拷贝初始化: string s= "value"
 - ◇ 直接初始化: string s (" value ")
 - ◇ 直接初始化相对好一点

▪ getline的函数

- 作用读取一整行的字符串, 一遇到换行符就停止读取
 - ◆ 注意这里是读到换行符（读取了）才停止对除了换行符以外的其他进行操作

▪ empty和size操作

- empty函数根据string对象是否为空返回一个对应的布尔值
- size函数返回string对象的长度（即string对象中字符的个数）, 可以使用size函数只输出长度超过80个字符的行
 - ◆ 注意
 - ◇ 如果一条表达式中已经有了size（）函数就不要再使用int了, 这样可以避免混用int和unsigned可能带来的问题。
 - ◇ 由于size函数返回的是一个无符号整型数, 因此切记, 如果在表达式中混用了带符号数和无符号数将可能产生意想不到的结果（参见2.1.2节, 第33页）。例如, 假设n是一个具有负值的int, 则表达式s.size（）<n的判断结果几乎肯定是true。这是因为负值n会自动地转换成一个比较大的无符号值。

```
string line;
// 每次读入一整行, 输出其中超过 80 个字符的行
□ while (getline(cin, line))
    if (line.size() > 80)
        cout << line << endl;
```

▪ > = 等符号的操作

- 相等性运算符（==和!=）分别检验两个string对象相等或不相等, string

对象相等意味着它们的长度相同而且所包含的字符也全都相同。关系运算符 <、<=、>、>= 分别检验一个 string 对象是否小于、小于等于、大于、大于等于另外一个 string 对象。上述这些运算符都依照（大小写敏感的）字典顺序

- ◆ 1. 如果两个 string 对象的长度不同，而且较短 string 对象的每个字符都与较长 string 对象对应位置上的字符相同，就说较短 string 对象小于较长 string 对象。
- ◆ 2. 如果两个 string 对象在某些对应的位置上不一致，则 string 对象比较的结果其实是 string 对象中第一对相异字符比较的结果。

▪ string 的操作

- 赋值：对于 string 类而言，允许把一个对象的值赋给另外一个对象

```
string s1 = "hello, ", s2 = "world\n";  
string s3 = s1 + s2;      // s3 的内容是 hello, world\n  
s1 += s2;                 // 等价于 s1 = s1 + s2
```

- 加法：

- ◆ （相当于 strcpy）对 string 对象使用加法运算符（+）的结果是一个新的 string 对象，它所包含的字符由两部分组成：前半部分是加号左侧 string 对象所含的字符、后半部分是加号右侧 string 对象所含的字符。

```
string st1(10, 'c'), st2; // st1 的内容是 cccccccccc; st2 是一个空字符串  
st1 = st2;               // 赋值：用 st2 的副本替换 st1 的内容  
                        // 此时 st1 和 st2 都是空字符串
```

- ◆ string 可以与字面值的类型相加，但是加字面值的时候一定也要有 string 类型的对象

```
string s4 = s1 + ", ";    // 正确：把一个 string 对象和一个字面值相加  
string s5 = "hello" + ", "; // 错误：两个运算对象都不是 string  
// 正确：每个加法运算符都有一个运算对象是 string  
string s6 = s1 + ", " + "world";  
string s7 = "hello" + ", " + s2; // 错误：不能把字面值直接相加
```

- ◆ 注意：因为某些历史原因，也为了与 C 兼容，所以 C++ 语言中的字符串字面值并不是标准库类型 string 的对象。切记，字符串字面值与 string 是不同的类型。

▪ 总结

- 字面值的两边至少要有有一个字符串才可以正常相加

○ ctype 标准库

- 主要是对于特定字符串中字符的处理

表 3.3: ctype 头文件中的函数	
isalnum(c)	当 c 是字母或数字时为真
isalpha(c)	当 c 是字母时为真
iscntrl(c)	当 c 是控制字符时为真
isdigit(c)	当 c 是数字时为真
isgraph(c)	当 c 不是空格但可打印时为真
islower(c)	当 c 是小写字母时为真
isprint(c)	当 c 是可打印字符时为真（即 c 是空格或 c 具有可视形式）
ispunct(c)	当 c 是标点符号时为真（即 c 不是控制字符、数字、字母、可打印空白中的一种）
isspace(c)	当 c 是空白时为真（即 c 是空格、横向制表符、纵向制表符、回车符、换行符、进纸符中的一种）
isupper(c)	当 c 是大写字母时为真
isxdigit(c)	当 c 是十六进制数字时为真
tolower(c)	如果 c 是大写字母，输出对应的小写字母；否则原样输出 c
toupper(c)	如果 c 是小写字母，输出对应的大写字母；否则原样输出 c

▪ 范围for语句

- 作用：处理每个字符
- 语法

for (*declaration : expression*)
statement

- 一般的，我们利用auto来确保每一个字符不会被漏掉

```
string s("Hello World!!!");
// punct_cnt 的类型和 s.size 的返回类型一样；参见 2.5.3 节（第 62 页）
decltype(s.size()) punct_cnt = 0;
//统计 s 中标点符号的数量
for (auto c : s)           // 对于 s 中的每个字符
    if (ispunct(c))        // 如果该字符是标点符号
        ++punct_cnt;      // 将标点符号的计数值加 1
cout << punct_cnt
     << " punctuation characters in " << s << endl;
```

- 如果想要改变string对象中字符的值，必须把循环变量定义成引用类型

◆ toupper是转换函数

```
string s("Hello World!!!");
// 转换成大写形式。
for (auto &c : s)           // 对于 s 中的每个字符（注意：c 是引用）
    c = toupper(c);        // c 是一个引用，因此赋值语句将改变 s 中字符的值
cout << s << endl;
```

- 简而言之：要计算数目，直接用auto定义，需要更改字符的值，使用引用
- 注意

◆ 范围for循环语句体内不应改变其所遍历序列的大小

- 范围for循环与普通for循环的区别

◆ 安全，范围for循环能保证循环在字节串之内游动不会出界，而普通for不行

▪ 访问string对象的两种方法

- 下标

- ◆ 定义：下标运算符 ([]) 接收的输入参数是string::size_type类型的值（参见3.2.2节，第79页），这个参数表示要访问的字符的位置；返回值是该位置上字符的引用。
- ◆ 注意

- ◇ string对象的下标从0计起。如果string对象s至少包含两个字符
- ◇ s[s.size () -1]是最后一个字符。
- ◇ string对象的下标必须大于等于0而小于s.size () 。
- ◇ 在访问指定字符之前，首先检查s是否为空。其实不管什么时候只要对string对象使用了下标，都要确认在那个位置上确实有值。如果s为空，则s[0]的结果将是未定义的。

◇

□ 迭代器

○ vector标准库

▪ vector是一个类模板

- 模板本身不是类或函数，相反可以将模板看作为编译器生成类或函数编写的一份说明。编译器根据模板创建类或函数的过程称为实例化 (instantiation) ， 当使用模板时，需要指出编译器应把类或函数实例化成何种类型。

▪ 类模板往往需要提供一些额外的信息用于模板的实体化

- 以vector为例，提供的额外信息是vector内所存放对象的类型：

```
vector<int> ivec;           // ivec 保存 int 类型的对象
vector<Sales_item> Sales_vec; // 保存 Sales_item 类型的对象
vector<vector<string>> file; // 该向量的元素是 vector 对象
```

▪ 注意

- 类模板就像是一个已经规划好的图纸，需要自己在进一步的说明就可以使用
- vector是模板而非类型，由vector生成的类型必须包含vector中元素的类型，例如vector<int>。
- vector能容纳绝大多数类型的对象作为其元素，但是因为引用不是对象（参见2.3.1节，第45页），所以不存在包含引用的vector。

□

▪ vector的操作

表 3.5: vector 支持的操作	
v.empty()	如果 v 不含有任何元素，返回真；否则返回假
v.size()	返回 v 中元素的个数
v.push_back(t)	向 v 的尾端添加一个值为 t 的元素
v[n]	返回 v 中第 n 个位置上元素的引用
v1 = v2	用 v2 中元素的拷贝替换 v1 中的元素
v1 = {a,b,c... }	用列表中元素的拷贝替换 v1 中的元素
v1 == v2	v1 和 v2 相等当且仅当它们的元素数量相同且对应位置的元素值都相同
v1 != v2	
<, <=, >, >=	顾名思义，以字典顺序进行比较

□

▪ 初始化vector对象

□ 举例

表 3.4: 初始化 vector 对象的方法	
vector<T> v1	v1 是一个空 vector，它潜在的元素是 T 类型的，执行默认初始化
vector<T> v2(v1)	v2 中包含有 v1 所有元素的副本
vector<T> v2 = v1	等价于 v2(v1)，v2 中包含有 v1 所有元素的副本
vector<T> v3(n, val)	v3 包含了 n 个重复的元素，每个元素的值都是 val
vector<T> v4(n)	v4 包含了 n 个重复地执行了值初始化的对象
vector<T> v5{a,b,c...}	v5 包含了初始值个数的元素，每个元素被赋予相应的初始值
vector<T> v5={a,b,c...}	等价于 v5{a,b,c...}

- ◆ 可以在定义vector对象时指定元素的初始值。例如，允许把一个vector对象的元素拷贝给另外一个vector对象。此时，新vector对象的元素就是原vector对象对应元素的副本。

```
vector<int> ivec;           // 初始状态为空
// 在此处给 ivec 添加一些值
vector<int> ivec2(ivec);    // 把 ivec 的元素拷贝给 ivec2
vector<int> ivec3 = ivec;   // 把 ivec 的元素拷贝给 ivec3
vector<string> svec(ivec2); // 错误: svec 的元素是 string 对象, 不是 int
```

- ◇ 注意两个对象类型要相同

- ◆ 列表初始化

- ◇ 举例

```
vector<string> articles = {"a", "an", "the"};
```

- ◇ C++11新标准还提供了另外一种为vector对象的元素赋初值的方法，即列表初始化（参见2.2.1节，第39页）。此时，用花括号括起来的0个或多个初始元素值被赋给vector对象：

□ 初始化vector的方法

- ◆ 直接初始化

- ◇ vector<int>{1,2,3}

- ◇ 意思：包含了三个int类型的元素，每个元素的值分别为1，2，3

- ◆ 列表初始化

- ◇ vector<int>={1,2,3}

- ◇ 意思：包含了三个int类型的元素，每个元素的值分别为1，2，3

- ◆ 值初始化

- ◇ vector<int>(10)

- ▶ 意思：包括了10个元素，每个元素都被值初始化（即为0）

- ◇ vector<int>(10,2)

- ▶ 意思：包括了10个元素，每个元素都被初始化值为2

- ◆ 容器之间的直接初始化

- ◇ vector<int>v1(v2)

- ◇ 意思：把v2所有的元素赋予v1

- ◆ 容器之间的间接初始化

- ◇ vector<int>v1=v2

- ◇ 意思：把v2所有的元素赋予v1

◆

□ 注意

- ◆ 我们使用花括号和圆括号来区分初始化的类别

- ◇ 对于值初始化，我们使用花括号，vector就会把他们认为是一个值

- ◇ 对于构造一定的对象，我们使用圆括号，vector就会给我们构造一定数量的对象

▪ 向vector中输入元素

□ 我们用函数push-back来输入元素

- ◆ 例如

```
vector<int> v2;
v2.push_back(2)//向着一个空的vector中输入2
```

□ 注意

- ◆ vector对象能高效的生长，所以一开始可以先创建一个空的vector，一开始先初始化vector会导致性能的损耗
- ◆ 对于一个“串”的类型，都能使用范围for循环
- ◆ 对于vector来说，当一个循环中有添加语句的句子时候，就不能使用范围for循环

▪ 读取vector中的元素

- 使用范围for循环读取元素
- 使用下标去读取元素

○ vector与string的相似之处

- vector是容器，string是类容器，因为string支持很多类似于vector的操作
- 在不更改大小的情况下，都可以使用范围for循环进行逐个输出
- 可以利用下标进行数据的查看与修改，但是不可以用下标新建一个数据（会报错）
- 下标的最大值都是x.size()-1，牢记于心
-

○ 迭代器

▪ 背景

- 所有的标准库容器都可以使用迭代器，但是只有少数几个才支持下标
- 迭代器类似于指针类型，迭代器也提供了对对象的间接访问

▪ 使用迭代器

□ 初始化迭代器

```
// 由编译器决定 b 和 e 的类型；参见 2.5.2 节（第 61 页）
// b 表示 v 的第一个元素，e 表示 v 尾元素的下一位置
auto b = v.begin(), e = v.end(); //b 和 e 的类型相同
```

- ◆ 其中b, e就为迭代器

□ 迭代器的操作

◆ 指南

表 3.5: vector 支持的操作	
v.empty()	如果 v 不含有任何元素，返回真；否则返回假
v.size()	返回 v 中元素的个数
v.push_back(t)	向 v 的尾端添加一个值为 t 的元素
v[n]	返回 v 中第 n 个位置上元素的引用
v1 = v2	用 v2 中元素的拷贝替换 v1 中的元素
v1 = {a,b,c...}	用列表中元素的拷贝替换 v1 中的元素
v1 == v2	v1 和 v2 相等当且仅当它们的元素数量相同且对应位置的元素值都相同
v1 != v2	
<, <=, >, >=	顾名思义，以字典顺序进行比较

◆ 迭代器类型

- ◇ 迭代器标准库使用iterator和const—iterator来表示迭代器的类型
- ◇ 如果vector对象或string对象是一个常量，只能使用const_iterator；如果vector对象或string对象不是常量，那么既能使用iterator也能使用const_iterator

◇ 注意

- ▶ 迭代器这个名词有三种不同的涵义，可能是迭代器本身，也可能是指容器定义的迭代器类型，还可能是指某个迭代器对象

◆ begin, end, cbegin, cend

- ◇ 对于begin和end，返回的对象都是可以修改的类型

▶ auto s=v.begin(); *s=xx

- ◇ 对于cbegin和cend，返回的对象都是不可以修改的

▶ auto s=v.cbegin(); (不能修改值)

◆ 结合解引用和访问成员操作

- ◇ 代码

▶ (*it).empty()

▶ it->empty

◆ 注意

- ◇ end迭代器返回的是最后那个元素的下一个元素，所以是没有实际意义的，因此也不能对end迭代器进行递增和递减的操作
- ◇ c++程序员要习惯使用!=，因为这种编程风格在标准库提供的所有容器中都有效
- ◇ 凡是使用了迭代器的循环体，都不要向迭代器所属的容器中添加元素

□ string, vector支持的特殊的迭代器运算操作

◆ 算数运算

表 3.7: vector 和 string 迭代器支持的运算	
<code>iter + n</code>	迭代器加上一个整数值仍得一个迭代器，迭代器指示的新位置与原来相比向前移动了若干个元素。结果迭代器或者指示容器内的一个元素，或者指示容器尾元素的下一位置
<code>iter - n</code>	迭代器减去一个整数值仍得一个迭代器，迭代器指示的新位置与原来相比向后移动了若干个元素。结果迭代器或者指示容器内的一个元素，或者指示容器尾元素的下一位置
<code>iter1 += n</code>	迭代器加法的复合赋值语句，将 <code>iter1</code> 加 <code>n</code> 的结果赋给 <code>iter1</code>
<code>iter1 -= n</code>	迭代器减法的复合赋值语句，将 <code>iter1</code> 减 <code>n</code> 的结果赋给 <code>iter1</code>
<code>iter1 - iter2</code>	两个迭代器相减的结果是它们之间的距离，也就是说，将运算符右侧的迭代器向前移动差值个元素后将得到左侧的迭代器。参与运算的两个迭代器必须指向的是同一个容器中的元素或者尾元素的下一位置
<code>>, >=, <, <=</code>	迭代器的关系运算符，如果某迭代器指向的容器位置在另一个迭代器所指位置之前，则说前者小于后者。参与运算的两个迭代器必须指向的是同一个容器中的元素或者尾元素的下一位置

- ◇ 注意这里+号是向右移动，-向左移动

- ◇ `iter1-iter2`可以理解为下标大的减去下标小的

○ 数组

▪ 字符数组

□ 定义数组

◆ 注意：不能使用变量定义数组

- ◇ 如n，函数表达式的返回值都不行

◆ 定义数组的三种方法

- ◇ 直接的数字定义

- ◇ 使用constexpr前缀定义，表示一个常量
 - ◇ 直接给定数字，让编译器自己去数
- 特别注意
 - ◆ 对于字符数组，每一个单词都是由一个字符组成的，最后加上 '\0'
 - ◆ 默认初始化的值为 '\0'
- 数组
 - 定义数组
 - ◆ 注意：不能使用变量定义数组
 - ◇ 如n，函数表达式的返回值都不行
 - ◆ 定义数组的三种方法
 - ◇ 直接的数字定义
 - ◇ 使用constexpr前缀定义，表示一个常量
 - ◇ 直接给定数字，让编译器自己去数
 - 初始化
 - ◆ 可以对数组进行列表初始化
 - ◆ 在默认状况下，数组被初始化为0
- 字符数组与字符串
 - 字符串数组
 - ◆ 字符串数组是用来储存一串字符的，而字符数组只是用来储存字符的
 - ◆ 字符串数组支持默认初始化
 - ◆ 字符串数组后面不用加 '\0'
- 注意
 - 对于所有数组，不允许数组的直接拷贝和赋值
 - 在使用数组的时候，我们可以使用for循环的使用最好用size_t，他的好处是他区别于机器码，是一个可以无限增大的数字
 - 两个数组相等的条件是两个数组的所有元素都相等
 - 尽量使用标准库类型而并非数组
 - ◆ 使用指针和数组很容易出错。一部分原因是概念上的问题
 - ◇ 指针常用于底层操作，因此容易引发一些繁琐细节的错误。其他问题则源于语法错误，特别是声明指针时候的语法错误
 - ◆ 现代c++应当尽量使用vector和迭代器，避免使用内置数组和指针，应当尽量使用string，避免使用c风格的基于数组的字符串
- 指针与数组
 - 复杂条件下指针与数组之间的交换
 - ◆ 要先进行去除小括号，然后从右向左依次读数
 - ◆ 数组也拥有迭代器
 - ◇ 标准库函数begin, end
 - ▶ 其中begin返回了开始的元素，end返回了尾部元素下一个元素的指针，这两个函数定义在iterator文件中

```
int ia【】={1, 2, 3, 4, 5, 6};
```

```
int beg=begin (ia) ;
```

```
int end=end (ia) ;
```

▪ 多维数组

- 注意，严格的来说，C++语言中没有多维数组，通常所说的多维数组其实是数组的数组。谨记这一点，对今后理解和使用多维数组大有益处

- 当一个数组的元素仍然是数组的时候，通常使用两个维度来定义它（初始化数组）

- ◆ 一个维度表示本身大小
- ◆ 另外一个维度表示其元素（也是数组的大小）
- ◆ 例如

◇ 数组的初始化

```
int ia[3][4]; // 大小为3的数组，每个元素是含有4个整数的数组
// 大小为10的数组，它的每个元素都是大小为20的数组，
// 这些数组的元素是含有30个整数的数组
int arr[10][20][30] = {0}; // 将所有元素初始化为0
```

```
int ia[3][4] = {           // 三个元素，每个元素都是大小为4的数组
    {0, 1, 2, 3},         // 第1行的初始值
    {4, 5, 6, 7},         // 第2行的初始值
    {8, 9, 10, 11}        // 第3行的初始值
};
```

// 没有标识每行的花括号，与之前的初始化语句是等价的

```
int ia[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

// 显式地初始化第1行，其他元素执行值初始化

```
int ix[3][4] = {0, 3, 6, 9};
```

// 显式地初始化每行的首元素

```
int ia[3][4] = {{ 0 }, { 4 }, { 8 }};
```

// 用arr的首元素为ia最后一行的最后一个元素赋值

```
ia[2][3] = arr[0][0][0];
```

```
int (&row)[4] = ia[1]; // 把row绑定到ia的第二个4元素数组上
```

□ 处理数组

- ◆ 可以使用嵌套循环来处理二维数组

◇ 范围for循环

- ▶ 要使用范围for语句处理多维数组，除了最内层的循环外，其他所有循环的控制变量都应该是引用类型

◇ 简单嵌套循环

□ 指针与多维数组

- ◆ 当程序使用多维数组的名字时候，也会在自动将其转换成指向数组首元素的指针

- ◆ 注意

- ◇ 定义指向多维数组的指针时，千万别忘了这个多维数组实际上是数组的数组

◇ 对于有指针的复杂类型，要先看小括号，最后从右向左边看

▪ 小结

- string 和vector是两种重要的标准库类型，string对象是一个可以变长的字符序列，vector对象是一组同类型对象的容器
- 迭代器类似于指针，他允许对容器中的对象进行间接的访问，对于string对象和vector对象来说，可以通过迭代器访问元素或者在元素之间进行移动
- 数组和指向数组元素的指针在一个较低的层次上实现了与标准库类型string和vector类似的功能。一般来说，应该优先选用标准库提供的类型，之后再考虑c++语言内置的低层的替代品数组或者指针

○ c++版本的c标准库头文件

- C++标准库中除了定义C++语言特有的功能外，也兼容了C语言的标准库。C语言的头文件形如name.h，C++则将这些文件命名为cname。也就是去掉了.h后缀，而在文件名name之前添加了字母c，这里的c表示这是一个属于C语言标准库的头文件。
- 因此，cctype头文件和ctype.h头文件的内容是一样的，只不过从命名规范上来讲更符合C++语言的要求。特别的，在名为cname的头文件中定义的名字从属于命名空间std，而定义在名为.h的头文件中的则不然。
- C标准库string函数

□ 混用string对象和c风格字符串

- ◆ string s (s有内容) ; const char *str=s. c_str () ---》str指向s的第一个串，反过来不行
- ◆ 如果执行完c—str () 函数后程序想一直都能使用其返回的数组，最好将该数组重新拷贝一份

□ 使用数组初始化vector对象（注意不是赋值，不是赋值，创建后就要初始化）

- ◆ int arr【】={0, 1, 2, 3, 4, 5, 6}; vector<int> ivec(begin(arr),end(arr))
- ◆ 需要告诉首尾地址
- ◆ 初始化的时候可以在begin和end后面加上数字，对一部分数组进行初始化

表 3.8: C 风格字符串的函数

strlen(p)	返回 p 的长度，空字符不计算在内
strcmp(p1, p2)	比较 p1 和 p2 的相等性。如果 p1==p2，返回 0；如果 p1>p2，返回一个正值；如果 p1<p2，返回一个负值
strcat(p1, p2)	将 p2 附加到 p1 之后，返回 p1
strcpy(p1, p2)	将 p2 拷贝给 p1，返回 p1